



MetaGPT 实战任务解析

API 模型: deepseek-reasoner (deepseek-v3.2 think)

框架: metagpt 框架, 通过 cli 调用

提示词:

1. 一个排序算法可视化网页
2. 一个网页版2048小游戏, 支持鼠标触控和键盘快捷键
3. 一个简单的多智能体框架 (python 语言), 使用 openapi 接口调用llm, 支持自定义智能体

项目1: 排序可视化

查看生成的文件夹, 发现 metagpt 框架创建了 (包含非代码文件) 的复杂文件结构, 但并不是每个文件夹内都有输出。

```
.
├── docs
│   ├── class_view
│   ├── code_plan_and_change
│   ├── code_summary
│   ├── graph_repo
│   ├── prd
│   │   └── 20251217144415.json
│   ├── requirement.txt
│   ├── system_design
│   │   └── 20251217144415.json
│   └── task
│       └── 20251217144415.json
├── requirements.txt
├── resources
│   ├── api_spec_and_task
│   ├── code_plan_and_change
│   ├── code_summary
│   ├── competitive_analysis
│   │   └── 20251217144415.mmd
│   ├── data_api_design
│   │   └── 20251217144415.mmd
│   ├── graph_db
│   ├── prd
│   │   └── 20251217144415.md
│   ├── sd_output
│   ├── seq_flow
│   │   └── 20251217144415.mmd
│   └── system_design
│       └── 20251217144415.md
├── sorting_visualizer
│   ├── css
│   │   └── style.css
│   ├── index.html
│   └── js
│       ├── algorithms
│       │   ├── BubbleSort.js
│       │   ├── InsertionSort.js
│       │   └── MergeSort.js
```

```
| | | QuickSort.js
| | | SelectionSort.js
| | | SortingAlgorithm.js
| | | controller
| | | PlaybackController.js
| | | UIController.js
| | | main.js
| | | utils
| | | EventEmitter.js
| | | visualizer
| | | ArrayData.js
| | | Visualizer.js
| | test_outputs
| | tests
```

29 directories, 24 files

prd文件

```
{"Language":"zh_cn","Programming Language":"JavaScript","Original Requirements":"一个排序算法可视化
```

System_design

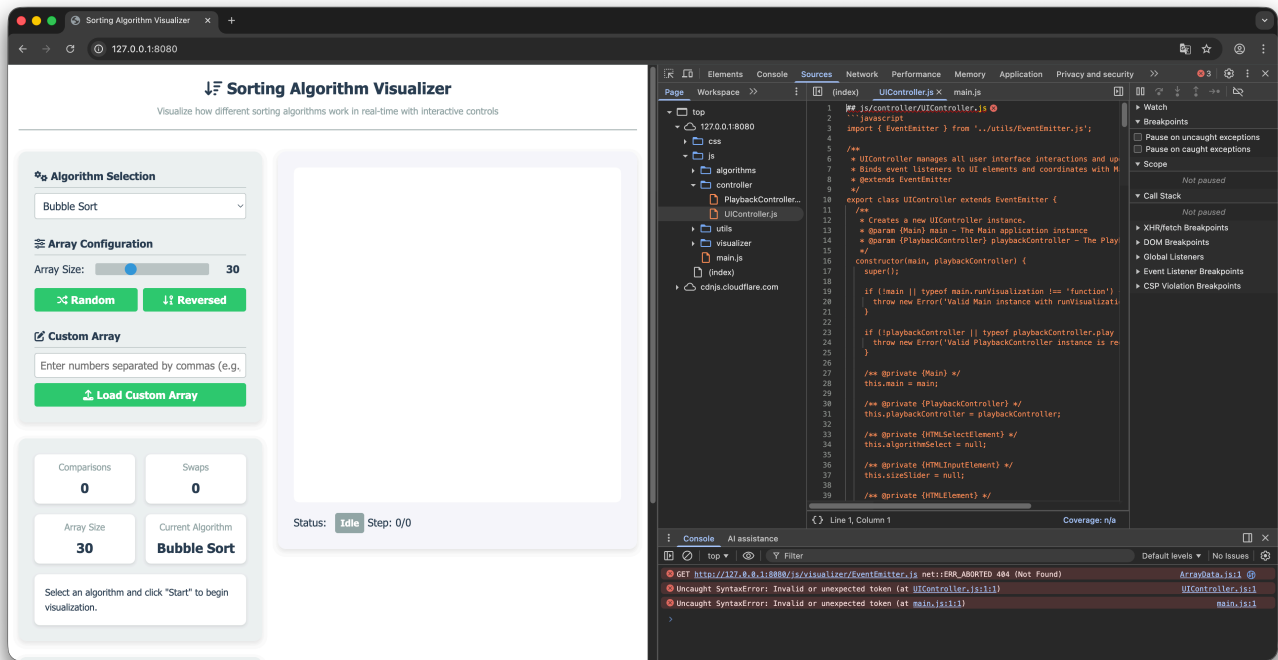
```
{"Implementation approach":"我们将构建一个纯客户端的、基于Web的排序算法可视化工具，无需后端。核心是使用原
```

task

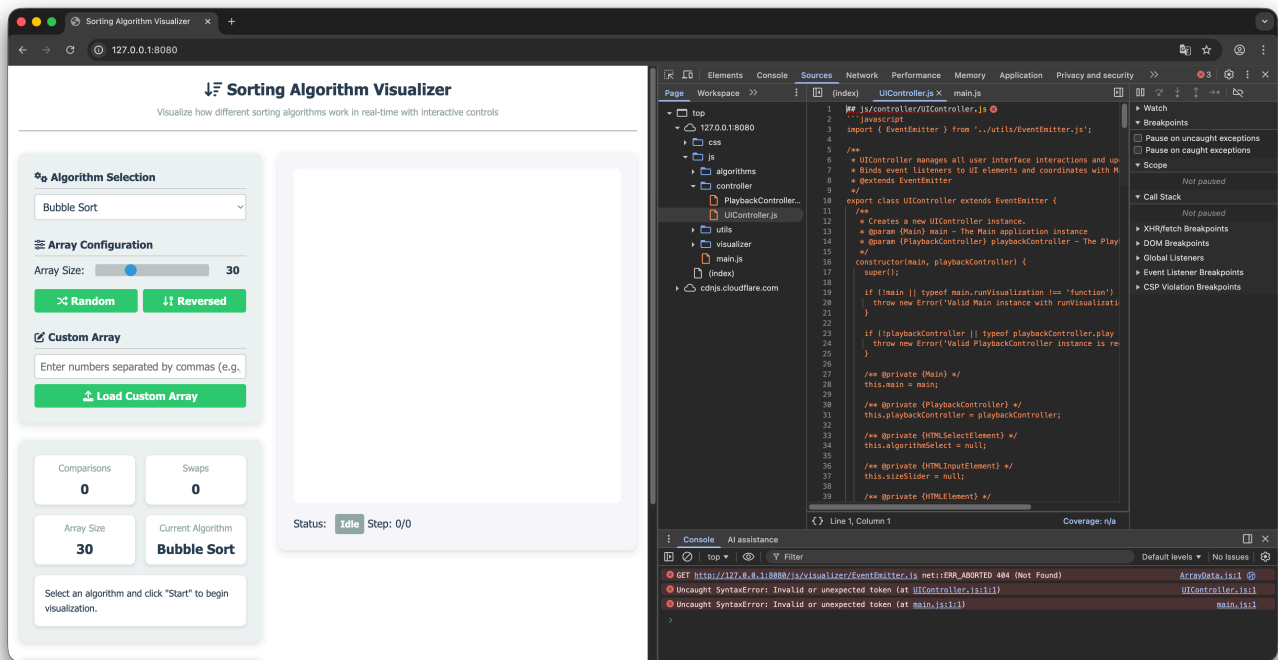
```
{"Required packages":["No third-party packages required. This is a pure front-end project using na
```

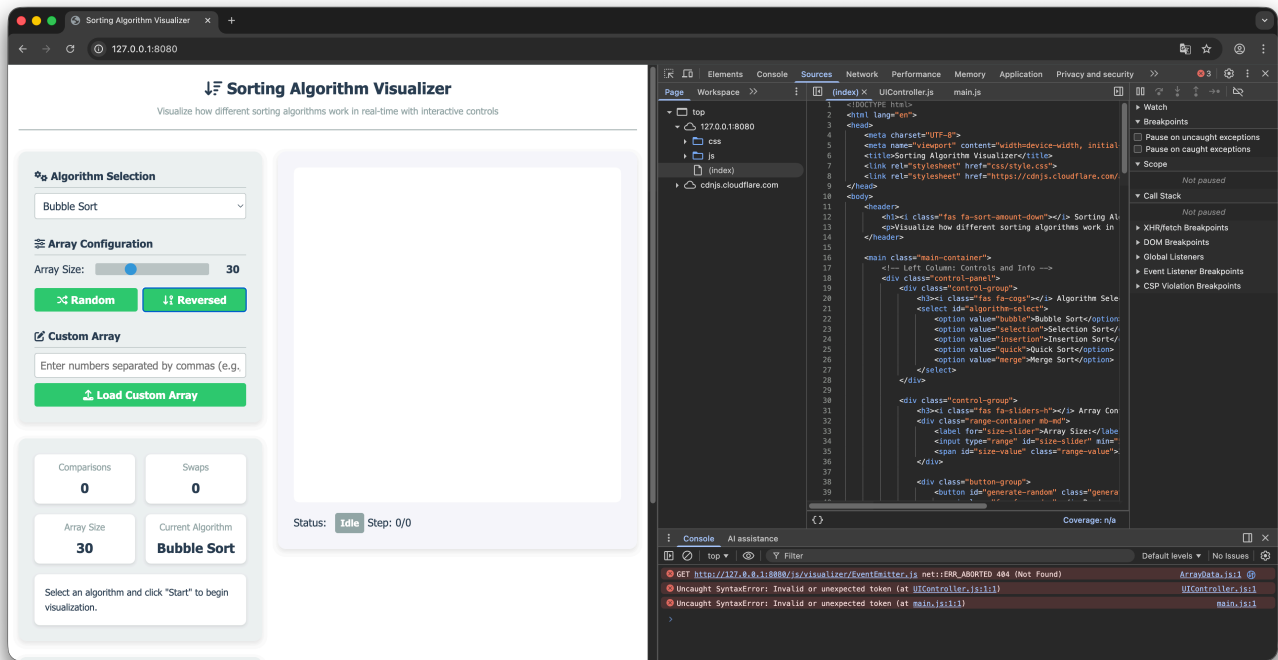
产出效果

使用 `http-server` 托管静态网页。使用 chrome 打开网页发现，网页可以看到具体的 UI 框架，但是具体的按钮、算法可视化功能不可用。打开浏览器工具，发现控制台提示报错，具体到对应的 js 文件发现，框架错误的把 markdown 内容写到了具体的 js 文件中。



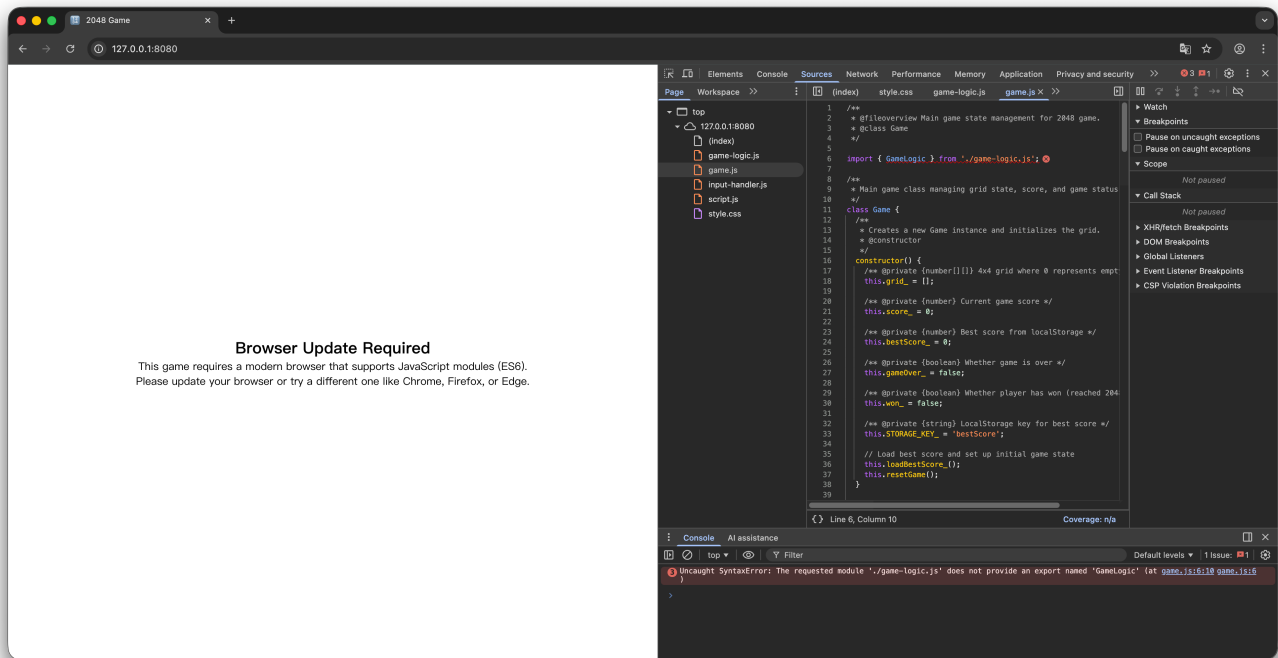
使用 `http-server` 托管静态网页。使用 chrome 打开网页发现，网页可以看到具体的 UI 框架，但是具体的按钮、算法可视化功能不可用。打开浏览器工具，发现控制台提示报错，具体到对应的 js 文件发现，框架错误的把 markdown 内容写到了具体的 js 文件中。





项目 2：2048

托管静态网页，项目完全不可用。其问题与项目 1 遇到的问题基本一致。



项目 3：agent 框架

框架优缺点分析

MetaGPT 的核心理念是 `Code = SOP(Team)`，在**多角色协作**、**SOP（标准操作流程）**、**软件开发场景**方面有独特优势。其工具推荐、计划管理等功能都是为支持复杂的多 agent 协作而设计的。总体而言，MetaGPT 已经具备了通用 agent 框架的核心能力，其相对"缺失"的部分主要集中在**可观测性**、**标准化服务接口**、**第三方集成生态**等工程化和生态建设方面，将 MetaGPT 的 agent 能力支持情况总结成表格如下：

核心能力	说明	MetaGPT 支持情况	
Tool Calling（工具调用）	调用外部工具和函数的能力	已支持	包括函数调用、工
Multi-Agent 协作	多个 Agent 之间的通信和协作	已支持	通过 Environment
ReAct 模式	Reasoning and Acting 的思考-行动循环	已支持	提供 react、by_d
Memory 管理	短期和长期记忆管理	已支持	支持短期记忆与长
Planning（规划）	任务分解和规划能力	已支持	内置 Planner 组件
Streaming（流式输出）	实时流式响应	已支持	在多个 provider
Callback/Hook 机制	监控和扩展钩子	相对薄弱	有 ResourceReport callback handler
Human-in-the-loop	人机交互能力	已支持	RoleZero 提供了
可观测性	完整的追踪和调试系统	相对薄弱	缺少完整调用链
工具推荐	动态推荐合适工具的能力	已支持	MetaGPT 具备动
标准化的 Agent-as-a-Service 接口	提供标准化服务接口（如 RESTful/gRPC）	相对薄弱	缺少 RESTful/gF

核心能力	说明	MetaGPT 支持情况	
丰富的第三方集成生态	对向量数据库、外部服务等集成丰富度	相对薄弱	集成选择较少，在
细粒度的权限和安全控制	工具调用权限、访问控制、审计日志等	相对薄弱	缺少完善的权限管